

Cahier des Charges

Plateforme de Gestion des Voyages d'Études

Projet web – HTML, CSS, JavaScript, PHP, MySQL

15 avril 2026

Table des matières

1	Présentation Générale du Projet	3
1.1	Périmètre général	3
1.2	Hors périmètre de la version initiale	3
2	Contexte et Problématique	4
3	Objectifs Globaux du Système	5
3.1	Objectifs principaux	5
3.2	Objectifs secondaires	5
4	Description Détaillée des Acteurs	6
4.1	Étudiant	6
4.2	Responsable Pédagogique	6
4.3	Administrateur	7
4.4	Service Financier (optionnel)	7
5	Analyse Fonctionnelle	8
5.1	Fonctionnalités Étudiant	8
5.2	Fonctionnalités Responsable Pédagogique	8
5.3	Fonctionnalités Administrateur	8
5.4	Fonctionnalités Service Financier (si implémenté)	9
5.5	Règles métier principales	9
6	Modélisation UML du Système	10
6.1	Diagramme de cas d'utilisation	10
6.2	Diagramme de classes	12
7	Cas d'Utilisation Principaux	14
7.1	Cas d'utilisation : Inscription d'un étudiant à un voyage	14
7.2	Cas d'utilisation : Création d'un voyage par un responsable	14
7.3	Cas d'utilisation : Validation d'un voyage par l'administrateur	15

8	Architecture Technique et Logicielle	16
8.1	Architecture globale	16
8.2	Architecture MVC (sans framework)	16
9	Modélisation Simplifiée de la Base de Données	17
9.1	Principes généraux	17
9.2	Table <code>users</code>	17
9.3	Table <code>voyages</code>	18
9.4	Table <code>inscriptions</code>	18
9.5	Table <code>documents</code>	18
9.6	Table <code>paiements</code> (optionnel)	18
10	Exigences Non Fonctionnelles	19
10.1	Sécurité (niveau adapté projet académique)	19
10.2	Accessibilité & Expérience Utilisateur	20
10.3	Performance	20
11	Contraintes Techniques	22
12	Diagramme Logique des Interactions	23
13	Perspectives d'Évolution	24
	Conclusion	25

Chapitre 1

Présentation Générale du Projet

Le présent document décrit le cahier des charges fonctionnel et technique de la **Plateforme de Gestion des Voyages d'Études**, une application web destinée à centraliser la gestion des voyages pédagogiques organisés par un établissement universitaire (visites industrielles, séjours culturels, projets scientifiques, échanges académiques).

1.1 Périmètre général

La solution repose sur une architecture web structurée :

- **Front-end** : HTML5, CSS3, JavaScript ;
- **Back-end** : PHP (architecture MVC maison, sans framework) ;
- **Base de données** : MySQL.

Le système couvre l'ensemble du cycle de vie d'un voyage d'études : proposition, validation, inscription des étudiants, suivi administratif et, de manière optionnelle, gestion des paiements.

1.2 Hors périmètre de la version initiale

Afin de garder un projet réaliste et faisable dans un cadre académique, les éléments suivants sont considérés comme hors périmètre pour la première version :

- paiement en ligne totalement automatisé ;
- application mobile native ;
- messagerie interne complète ;
- génération avancée de rapports analytiques ;
- intégration avec les systèmes externes de l'établissement.

Chapitre 2

Contexte et Problématique

Dans de nombreux établissements, la gestion des voyages d'études est encore réalisée via :

- des fichiers Excel ;
- des échanges d'emails ;
- des formulaires papier ;
- des listes manuelles.

Cela entraîne plusieurs conséquences :

- perte d'informations ou doublons ;
- manque de visibilité sur les inscriptions et les places disponibles ;
- processus longs et peu automatisés ;
- difficulté à tracer les décisions (qui a validé quoi, quand ?) ;
- communication peu structurée entre les différents acteurs (administration, responsables, étudiants).

Problématique principale :

Comment proposer un outil unique, structuré et sécurisé permettant de gérer de manière centralisée les voyages d'études d'un établissement, depuis leur création jusqu'au suivi des participants ?

Chapitre 3

Objectifs Globaux du Système

3.1 Objectifs principaux

Les objectifs principaux du système sont les suivants :

- centraliser la gestion des voyages d'études au sein d'une plateforme unique ;
- automatiser les tâches répétitives (inscriptions, validations, listes, notifications) ;
- assurer la traçabilité des décisions, des inscriptions et des paiements éventuels ;
- garantir un minimum de sécurité (authentification, gestion des rôles, protection basique des données) ;
- offrir une expérience utilisateur claire et adaptée aux différents profils (étudiants, responsables, administrateurs).

3.2 Objectifs secondaires

Les objectifs secondaires sont :

- réduction du temps de traitement administratif ;
- réduction des erreurs humaines liées aux saisies manuelles ;
- mise à disposition de statistiques simples (nombre de voyages, taux de remplissage, etc.) ;
- base technique suffisamment propre pour permettre des évolutions futures (paiement en ligne complet, application mobile, etc.).

Chapitre 4

Description Détaillée des Acteurs

4.1 Étudiant

Rôle : utilisateur final principal, bénéficiaire du voyage.

Responsabilités principales :

- créer un compte et se connecter ;
- consulter la liste des voyages disponibles ;
- consulter les détails d'un voyage ;
- s'inscrire à un voyage ;
- suivre l'état de son inscription (en attente, validée, refusée) ;
- fournir les documents nécessaires (scan de documents, autorisations, etc.) ;
- (optionnel) effectuer ou confirmer un paiement.

4.2 Responsable Pédagogique

Rôle : enseignant ou responsable de filière organisant le voyage.

Responsabilités principales :

- proposer et créer de nouveaux voyages d'études ;
- modifier les informations des voyages (dates, nombre de places, description, etc.) ;
- consulter la liste des étudiants inscrits ;
- valider ou refuser certaines inscriptions ;
- communiquer les informations importantes aux participants (via la plateforme, selon les fonctionnalités retenues).

4.3 Administrateur

Rôle : gestion globale du système.

Responsabilités principales :

- gérer les comptes utilisateurs (création, suppression, changement de rôle) ;
- valider ou refuser les voyages proposés par les responsables pédagogiques ;
- gérer certains paramètres globaux de l'application ;
- consulter des statistiques générales ;
- archiver les voyages passés.

4.4 Service Financier (optionnel)

Rôle : suivi des paiements relatifs aux voyages.

Responsabilités principales :

- consulter la liste des paiements associés aux inscriptions ;
- valider manuellement un paiement (si le paiement en ligne n'est pas automatisé) ;
- exporter ou consulter un rapport synthétique des montants.

Cet acteur peut être conservé dans le modèle, même si la partie paiement est simplifiée au départ. Si la version initiale ne comprend pas le suivi financier, cet acteur et les objets liés peuvent être retirés du périmètre sans remettre en cause le reste du système.

Chapitre 5

Analyse Fonctionnelle

5.1 Fonctionnalités Étudiant

- création de compte (inscription) ;
- authentification (connexion / déconnexion) ;
- gestion minimale du profil (email, mot de passe) ;
- consultation de la liste des voyages disponibles ;
- filtrage / recherche basique de voyages (par destination, date, etc.) ;
- consultation du détail d'un voyage (description, dates, budget, nombre de places restantes, etc.) ;
- inscription à un voyage ;
- visualisation de l'historique de ses inscriptions ;
- téléversement de documents liés au voyage (optionnel).

5.2 Fonctionnalités Responsable Pédagogique

- connexion sécurisée ;
- création d'un voyage d'études ;
- modification et mise à jour des informations d'un voyage ;
- annulation / désactivation d'un voyage ;
- consultation de la liste des étudiants inscrits par voyage ;
- validation / refus de certaines inscriptions (en fonction des critères pédagogiques).

5.3 Fonctionnalités Administrateur

- connexion sécurisée ;

- gestion des comptes utilisateur (création, activation/désactivation, changement de rôle : étudiant / responsable / admin) ;
- validation / refus des voyages proposés par les responsables ;
- consultation de listes globales :
 - voyages en attente de validation ;
 - voyages validés ;
 - voyages archivés.
- accès à des statistiques simples :
 - nombre de voyages par année ;
 - nombre d'inscrits par voyage.
- (optionnel) gestion de certains paramètres généraux (limite de places par défaut, messages d'information, etc.).

5.4 Fonctionnalités Service Financier (si implémenté)

- connexion ;
- consultation de la liste des paiements enregistrés ;
- validation manuelle d'un paiement (par exemple après vérification d'un virement) ;
- export ou affichage d'un résumé des paiements par voyage.

5.5 Règles métier principales

Les règles métier ci-dessous permettent de lever les ambiguïtés et de faciliter l'implémentation :

- un étudiant ne peut s'inscrire qu'à un voyage **validé** et **ouvert** ;
- une inscription est refusée automatiquement si le voyage est complet ;
- un étudiant ne doit pas pouvoir s'inscrire deux fois au même voyage ;
- un responsable peut créer un voyage, mais la publication effective dépend de la validation par l'administrateur ;
- un voyage peut être archivé lorsqu'il est terminé ;
- les documents déposés doivent être associés à un utilisateur identifié ;
- les actions sensibles doivent être réservées aux rôles autorisés.

Chapitre 6

Modélisation UML du Système

La modélisation UML du système reposera principalement sur :

- un **diagramme de cas d'utilisation** (Use Case) ;
- un **diagramme de classes**.

6.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation représentera :

- les quatre acteurs : Étudiant, Responsable pédagogique, Administrateur, Service financier (optionnel) ;
- les cas d'utilisation principaux, parmi lesquels (exemples) :
 - Créer un compte, se connecter, consulter les voyages, s'inscrire à un voyage (Étudiant) ;
 - Créer un voyage, modifier un voyage, consulter les inscriptions, valider les inscriptions (Responsable) ;
 - Gérer les utilisateurs, valider un voyage, consulter les statistiques (Administrateur) ;
 - Consulter les paiements, valider un paiement (Service financier).

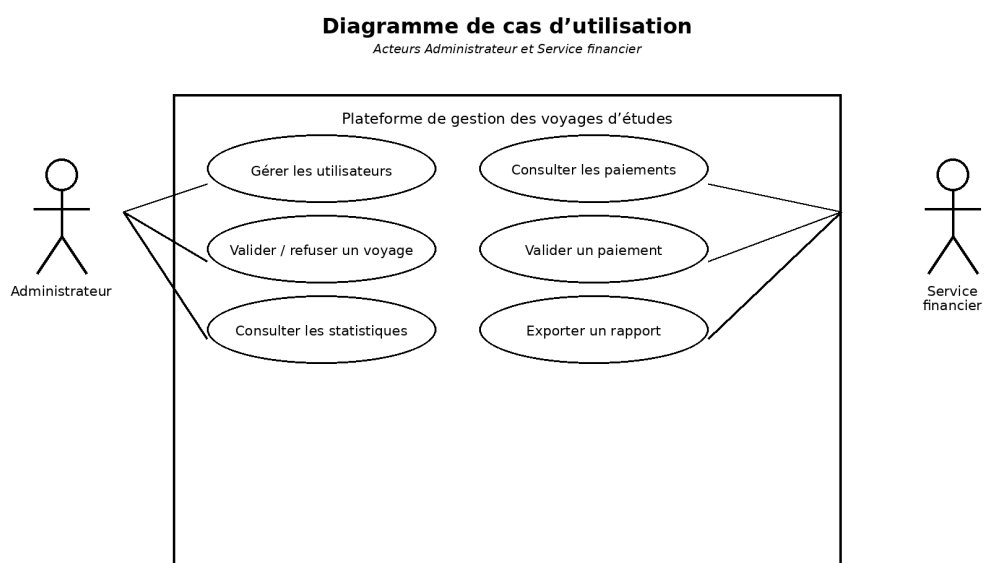


FIGURE 6.1 – Diagramme de cas d'utilisation – Acteurs Administrateur et Service financier

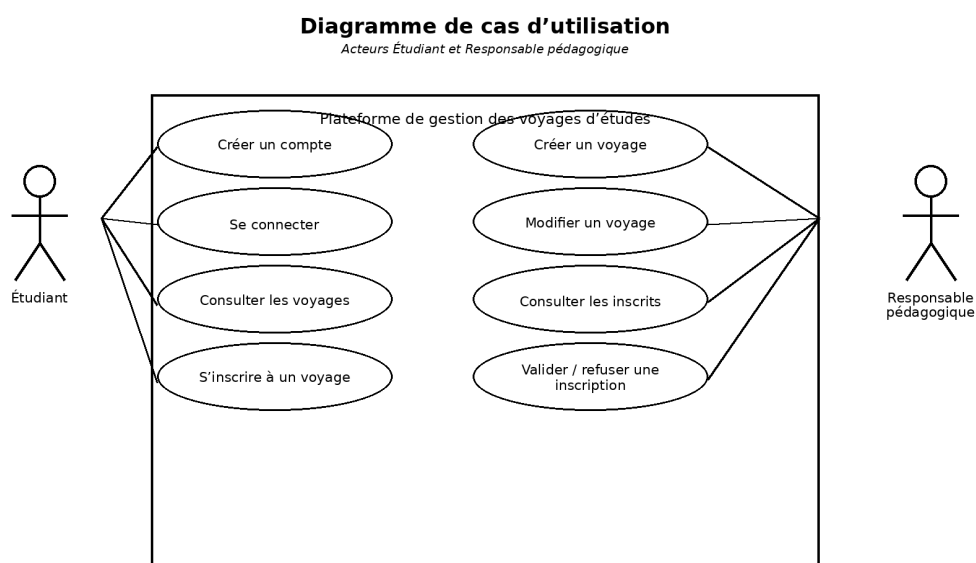


FIGURE 6.2 – Diagramme de cas d'utilisation – Acteurs Étudiant et Responsable pédagogique

Description du diagramme de cas d'utilisation Le diagramme de cas d'utilisation illustre les interactions entre les différents acteurs du système et les fonctionnalités principales offertes par la plateforme. Il permet de visualiser les services accessibles selon le rôle de l'utilisateur.

L'acteur *Étudiant* interagit principalement avec les fonctionnalités liées à la consultation des voyages, à l'inscription et au suivi de ses demandes. Le *Responsable pédagogique* dispose de fonctionnalités de gestion des voyages et de validation des inscriptions. L'*Administrateur* assure la gestion globale du système, notamment la gestion des utilisateurs et la validation des voyages proposés. Enfin, le *Service financier*, lorsqu'il est intégré, intervient dans le suivi et la validation des paiements.

Ce diagramme permet de structurer l'analyse fonctionnelle en mettant en évidence la répartition claire des responsabilités et des droits d'accès selon les profils d'utilisateurs.

6.2 Diagramme de classes

Un diagramme de classes décrira la structure logique des principales entités métier :

- **User**
- **Voyage**
- **Inscription**
- **Document**
- **Paielement** (optionnel)

Avec des associations du type :

- un **User** (responsable) peut être lié à plusieurs **Voyage** ;
- un **Voyage** peut avoir plusieurs **Inscription** ;
- un **User** (étudiant) peut avoir plusieurs **Inscription** ;
- une **Inscription** peut être liée à un **Paielement** ;
- un **User** peut avoir plusieurs **Document**.

Les attributs et méthodes principaux seront détaillés (par exemple `estComplet()`, `valider()`, etc.).

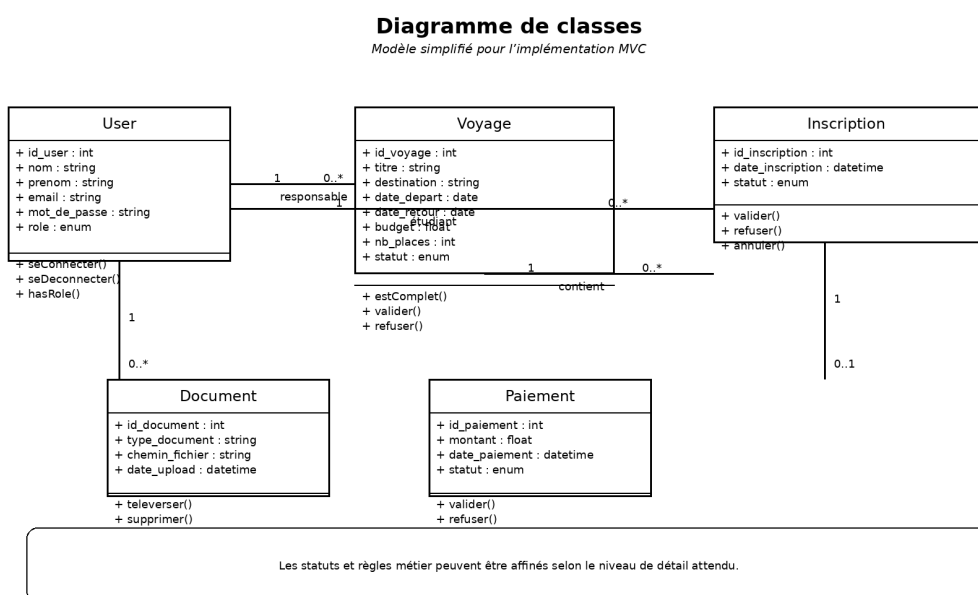


FIGURE 6.3 – Diagramme de classes

Description du diagramme de classes Le diagramme de classes présenté ci-dessus illustre la structure statique du système ainsi que les principales entités métier intervenant dans la plateforme de gestion des voyages d'études. Il met en évidence les classes fondamentales telles que **User**, **Voyage**, **Inscription**, **Document** et **Paiement**, ainsi que leurs attributs et méthodes principales.

La classe **User** représente l'ensemble des utilisateurs du système (étudiant, responsable pédagogique, administrateur, service financier), le rôle étant déterminé par un attribut spécifique. La classe **Voyage** modélise les informations relatives aux voyages proposés, tandis que la classe **Inscription** assure la liaison entre un étudiant et un voyage donné.

Les associations entre les classes traduisent les relations métier : un responsable peut créer plusieurs voyages, un voyage peut comporter plusieurs inscriptions, et un étudiant peut s'inscrire à plusieurs voyages. La présence éventuelle d'un **Paiement** lié à une **Inscription** permet d'étendre le système à la gestion financière. Ce diagramme constitue la base conceptuelle de l'implémentation en architecture MVC.

Chapitre 7

Cas d'Utilisation Principaux

7.1 Cas d'utilisation : Inscription d'un étudiant à un voyage

1. L'étudiant se connecte à la plateforme.
2. Il accède à la liste des voyages disponibles.
3. Il sélectionne un voyage et affiche ses détails.
4. Il clique sur le bouton « S'inscrire ».
5. Le système vérifie :
 - que l'étudiant est authentifié ;
 - que le voyage est validé et encore ouvert ;
 - que le nombre de places n'est pas dépassé ;
 - que l'étudiant n'est pas déjà inscrit à ce voyage.
6. Si tout est correct, une inscription est créée avec le statut « en attente » ou « enregistrée » selon la logique retenue.
7. Un message de confirmation est affiché à l'étudiant.

7.2 Cas d'utilisation : Création d'un voyage par un responsable

1. Le responsable pédagogique se connecte.
2. Il accède à la rubrique « Mes voyages » puis à l'option « Créer un voyage ».
3. Il remplit le formulaire (titre, destination, dates, description, nombre de places, budget, etc.).
4. Il envoie la demande de création.

5. Le système enregistre le voyage avec un statut « en attente de validation admin ».
6. L'administrateur pourra ensuite le valider ou le refuser.

7.3 Cas d'utilisation : Validation d'un voyage par l'administrateur

1. L'administrateur se connecte.
2. Il consulte la liste des voyages en attente.
3. Il affiche les détails d'un voyage proposé.
4. Il choisit de valider ou de refuser le voyage.
5. Le système met à jour le statut du voyage.
6. Le responsable peut alors voir le statut mis à jour et les étudiants pourront s'inscrire si le voyage est validé.

Chapitre 8

Architecture Technique et Logicielle

8.1 Architecture globale

La plateforme adopte une architecture **3-tiers** :

- couche Présentation (Front-end) ;
- couche Logique métier (Back-end PHP – MVC maison) ;
- couche Données (MySQL).

8.2 Architecture MVC (sans framework)

Le back-end sera organisé selon une architecture inspirée du modèle MVC :

Model (M) Représente les entités métier : **User**, **Voyage**, **Inscription**, etc. Il contient la logique d'accès aux données (requêtes SQL via PDO) et encapsule les opérations CRUD sur chaque entité.

View (V) Fichiers PHP/HTML responsables de l'affichage des pages : formulaires, listes, tableaux, messages. Ils intègrent également le CSS et le JavaScript (ou leurs liens).

Controller (C) Fichiers PHP recevant les requêtes HTTP (par exemple `voyageController.php`, `UserController.php`). Ils appellent les modèles appropriés, traitent la logique métier, puis sélectionnent la vue à afficher.

Organisation possible des dossiers :

- `public/` : point d'entrée, fichiers accessibles directement ;
- `app/controllers/` ;
- `app/models/` ;
- `app/views/` ;
- `app/config/` : fichier de connexion à la base, paramètres ;
- `assets/css/`, `assets/js/`, `assets/img/`.

Chapitre 9

Modélisation Simplifiée de la Base de Données

9.1 Principes généraux

La base de données doit refléter les règles métier du système et rester suffisamment simple pour un projet académique. Les contraintes suivantes sont recommandées :

- clés primaires sur chaque table ;
- contraintes d'unicité sur `users.email` ;
- contrainte d'unicité composée sur `inscriptions(id_user, id_voyage)` afin d'empêcher les doublons ;
- clés étrangères pour assurer l'intégrité référentielle ;
- index sur les colonnes fréquemment filtrées (`id_voyage, id_user, role, statut`) ;
- champs de date de création et, si besoin, de mise à jour.

9.2 Table `users`

- `id_user` (PK, int, auto)
- `nom` (varchar)
- `prenom` (varchar)
- `email` (varchar, unique)
- `mot_de_passe` (varchar, mot de passe hashé)
- `role` (enum : « étudiant », « responsable », « admin », éventuellement « financier »)
- `date_creation` (datetime)

9.3 Table voyages

- id_voyage (PK)
- titre (varchar)
- destination (varchar)
- description (text)
- date_depart (date/datetime)
- date_retour (date/datetime)
- budget (float)
- nb_places (int)
- statut (enum : « en_attente », « valide », « refuse », « archive »)
- id_responsable (FK → users.id_user)

9.4 Table inscriptions

- id_inscription (PK)
- id_user (FK → users.id_user)
- id_voyage (FK → voyages.id_voyage)
- date_inscription (datetime)
- statut (enum : « en_attente », « valide », « refuse », « annule »)

9.5 Table documents

- id_document (PK)
- id_user (FK → users.id_user)
- type_document (varchar)
- chemin_fichier (varchar)
- date_upload (datetime)

9.6 Table paiements (optionnel)

- id_paiement (PK)
- id_inscription (FK → inscriptions.id_inscription)
- montant (float)
- date_paiement (datetime)
- statut (enum : « en_attente », « valide », « refuse »)

Chapitre 10

Exigences Non Fonctionnelles

10.1 Sécurité (niveau adapté projet académique)

Les aspects de sécurité sont pris en compte à un niveau réaliste pour un projet académique :

Authentification sécurisée

- mots de passe stockés avec `password_hash()` ;
- vérification avec `password_verify()`.

Protection contre l'injection SQL

- utilisation de PDO avec requêtes préparées (*prepared statements*) ;
- aucune concaténation directe de valeurs utilisateur dans les requêtes SQL.

Gestion des sessions

- utilisation de `session_start()` ;
- pages sensibles accessibles uniquement si l'utilisateur est authentifié ;
- gestion des rôles (accès admin / responsable / étudiant).

Validation des données côté serveur

- contrôles sur les champs obligatoires ;
- vérification de la cohérence (dates, nombre de places, etc.) ;
- contrôle des fichiers téléversés (type, taille, extension).

Protection complémentaire recommandée

- protection CSRF sur les formulaires sensibles ;
- messages d'erreur non révélateurs ;
- journalisation minimale des actions importantes.

À noter : ce niveau reste pédagogique, sans viser un niveau de conformité industrielle complet, mais montre une compréhension des bonnes pratiques de base.

10.2 Accessibilité & Expérience Utilisateur

L'accessibilité est également abordée de façon adaptée :

HTML sémantique

- utilisation de balises structurantes : `header`, `nav`, `main`, `section`, `footer`.

Formulaires accessibles

- association systématique `<label>` / `<input>`.

Images

- attribut `alt` pour toutes les images importantes.

Responsive design

- utilisation de flexbox / grid ;
- ajout de la balise `<meta name="viewport" ...>`.

Lisibilité

- choix de couleurs contrastées suffisantes pour le texte ;
- taille de police lisible sur écran d'ordinateur et smartphone.

L'objectif est d'offrir une interface utilisable et agréable, sans prétendre couvrir toutes les normes WCAG de manière exhaustive.

10.3 Performance

- pagination basique sur les listes longues (voyages, inscriptions) ;
- limitation du nombre de résultats affichés par page ;

- indexation sur les colonnes fréquemment utilisées dans les recherches (`id`, `role`, `id_voyage`, etc.).

Chapitre 11

Contraintes Techniques

- hébergement supportant :
 - PHP 8+ ;
 - MySQL 8+.
- compatibilité navigateurs modernes (Chrome, Firefox, Edge) ;
- déploiement initial sur un environnement de type serveur local (XAMPP, WAMP, etc.) puis possibilité de migration vers un hébergement web classique ;
- organisation du code permettant la maintenance (fichiers séparés, commentaires, noms de fichiers explicites).

Chapitre 12

Diagramme Logique des Interactions

Flux général d'une requête :

1. L'utilisateur saisit une URL ou effectue une action (clic sur un bouton, soumission de formulaire).
2. La requête est reçue par le serveur HTTP (Apache/Nginx) qui redirige vers un contrôleur PHP.
3. Le contrôleur analyse la requête (paramètres GET/POST, session, etc.).
4. Le contrôleur appelle les modèles nécessaires (**User**, **Voyage**, **Inscription**, etc.) pour :
 - récupérer des données depuis MySQL ;
 - ou enregistrer / modifier des données.
5. Les données retournées par les modèles sont préparées par le contrôleur.
6. Le contrôleur choisit la vue (template PHP/HTML) à afficher et lui transmet les données.
7. La vue est rendue (HTML + CSS + JS) et renvoyée au navigateur.

Schéma logique textuel :

Navigateur → *Contrôleur* → *Modèle(s)* → *Base de données MySQL* → *Modèle(s)* →
Contrôleur → *Vue* → *Navigateur*

Chapitre 13

Perspectives d'Évolution

- intégration d'un vrai module de paiement en ligne (API bancaire ou service tiers) ;
- module de messagerie interne (responsable $\leftrightarrow$ étudiants) ;
- génération automatique de documents PDF (liste des participants, convocations, etc.) ;
- ajout d'un tableau de bord analytique (statistiques détaillées) ;
- développement d'une application mobile ou d'une PWA pour l'accès simplifié sur smartphone ;
- intégration avec d'autres systèmes de l'établissement (ex. annuaire des étudiants).

Conclusion

La plateforme de gestion des voyages d'études proposée présente une cohérence globale tant sur le plan fonctionnel que technique. L'analyse des besoins, la définition précise des acteurs et la modélisation UML assurent une correspondance claire entre les exigences métier et la structure du système.

Les choix architecturaux, notamment l'adoption d'une architecture trois tiers et d'un modèle MVC, garantissent une séparation logique des responsabilités, favorisant la maintenabilité, l'évolutivité et la clarté du code.

La modélisation de la base de données, alignée avec le diagramme de classes, ainsi que la définition des exigences non fonctionnelles (sécurité, accessibilité, performance) démontrent que le système est techniquement structuré et prêt à être implémenté dans un environnement PHP/MySQL. L'ensemble constitue ainsi une base solide pour le développement effectif de l'application.